Predicates

The '-any?' function takes a predicate function and a list, and returns 'true' if any member in the list satisfies the predicate. A value 'nil' is returned if there is no element that satisfies the predicate function. For example:

```
(-any? predicate list);; Syntax

(defun even? (num) (= 0 (% num 2)))

(-any? 'even? '(1 2 3))
t
```

If you would like to check if all the elements of the list match the predicate function, you can use the '-all?' function, as shown below:

```
(-all? predicate list) ;; Syntax

(-all? 'even? '(1 2 3))
nil
```

The '-none?' function returns 'true' (t) if the predicate function for all the members of the list returns 'nil'. It returns 'nil' otherwise. A couple of examples are given below:

```
(-none? predicate list);; Syntax

(-none? 'even? '(1 2 3))
nil

(-none? 'even? '(3 5 7))
t
```

The '-only-some?' function returns 'true' if at least one element in the list satisfies the predicate function, and one element does not match the predicate function. A few examples are shown below for reference:

```
(-only-some? predicate list);; Syntax


(-only-some? 'even? '(1 2 3))
t


(-only-some? 'even? '(2 4 6))
nil
```

If you would like to check whether an element is present in a list or not, you can use the '-contains?' function as illustrated below:

```
(-contains? list element);; Syntax

(-contains? '(1 2 3) 1)
t

(-contains? '(1 2 3) 5)
nil
```

A couple of lists can be compared to see if they have the same elements using the '-same-items?' function. The order of the elements in the list is not a concern for this function. A few examples of the function's use are given below:

```
(-same-items? list1 list2);; Syntax

(-same-items? '(4 5 6) '(4 5 6))
t

(-same-items? '(4 5 6) '(6 5 4))
t

(-same-items? '(4 5 6) '(6 5 4 3))
nil
```

The '-is-prefix?' function takes two arguments, - *a* prefix list and an input list. This function returns 'true' if the input list begins with the prefix list. Similarly, there exists '-is-suffix?' and '-is-infix?' functions to check for suffix and infix positions, respectively. A few examples are shown below:

```
(-is-prefix? prefix list);; Syntax
(-is-suffix? suffix list);; Syntax
(-is-infix?  infix  list);; Syntax

(-is-prefix? '(1 2 3) '(3 4 5))
nil

(-is-prefix? '(1 2) '(1 2 3))
t

(-is-suffix? '(2 3) '(1 2 3))
t

(-is-suffix? '(3 5) '(1 2 3 4 5))
nil

(-is-infix? '(1 3) '(1 2 3 4 5))
nil

(-is-infix? '(2 3) '(1 2 3 4 5))
t
```

Partitioning

The partitioning functions exist to break an input list into multiple lists. The '-split-at' function splits an input list at a position. For example: